

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Мобильная разработка

Отчет
Курсовая работа

Выполнил:

Якунин Андрей

Группа

К3441

Проверил:

Шатинский Г.С.

Санкт-Петербург
2026 г.

Отчёт по курсовой работе

Тема

Разработка Android-приложения «Мессенджер» с использованием встроенной навигации и архитектуры MVVM

Стек: Kotlin, Android SDK, AndroidX, Navigation Component, ViewModel и LiveData, Room, Retrofit, Material Components for Android.

Задача 1 - навигация по вкладкам

ТЗ

В рамках работы требуется разработать простое Android-приложение «Мессенджер» с одной главной Activity и навигацией на основе Fragment. Навигация между экранами реализуется с помощью Bottom Navigation и компонента Navigation. Приложение должно содержать экраны новостей, профиля и настроек. Также необходимо реализовать логирование жизненного цикла Activity и Fragment.

Решение

MainActivity - главная Activity с Bottom Navigation

Navigation Graph - настроена навигация между 3 экранами

Три фрагмента:

FeedFragment (новостная лента - заглушка)

ProfileFragment (профиль пользователя с данными)

SettingsFragment (переключатель темной/светлой темы)

NavController - управление навигацией через Navigation Component

Логирование жизненного цикла - Log.d() в onCreate, onStart, onResume, onPause, onStop, onDestroy для Activity и всех Fragments

Вывод:

Было сделано 3 экрана: Лента, Профиль, Настройка

Экран профиля сделан для отображения и редактирования пользовательских данных, таких как имя и статус. Данные доступны через LiveData

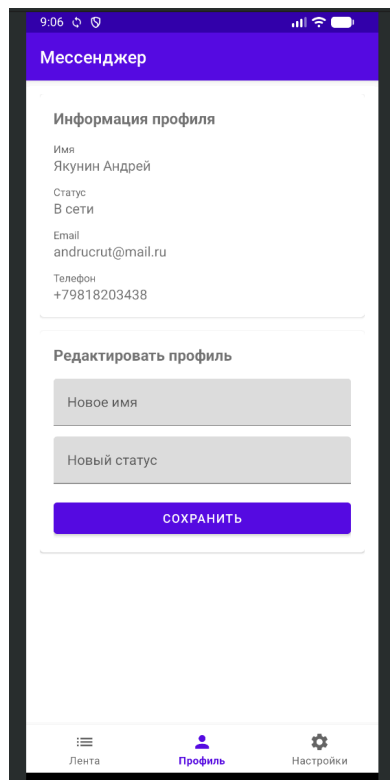


Рисунок 1 - Экран профиля

На экране указана актуальная информация профиля, а также возможность редактировать данные(имя, статус)

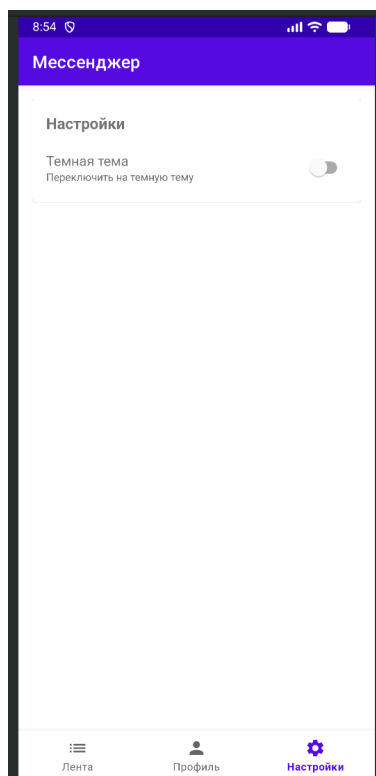


Рисунок 2 Экран настроек

Экран настроек имеет простой и лаконичный интерфейс, в котором представлены только те элементы, которые необходимы для работы с настройками. Центральное место занимает переключатель (Switch), с помощью которого пользователь может выбрать режим оформления приложения - светлый или тёмный

Задача 2 - MVVM и управление состоянием профиля и настроек

ТЗ

Необходимо реализовать архитектуру MVVM для управления состоянием приложения. Данные профиля пользователя и настройки темы должны храниться во ViewModel и сохраняться при повороте экрана. Экран профиля позволяет редактировать имя и статус, а экран настроек - переключать тему оформления. Обновление интерфейса выполняется реактивно через LiveData с логированием жизненного цикла ViewModel

Решение

1) Использование ViewModel для хранения состояния.

Для экранов профиля и настроек реализованы отдельные ViewModel, в которых хранятся данные пользователя и выбранные параметры оформления. Такой подход позволяет вынести логику работы с данными за пределы UI, оставив интерфейсу только отображение состояния и обработку пользовательских действий.

2) Экран профиля пользователя.

На экране профиля отображаются имя и статус пользователя, которые можно изменять с помощью полей ввода. Введённые значения сохраняются во ViewModel, а интерфейс автоматически обновляется при изменении данных благодаря использованию LiveData. Это обеспечивает корректную работу экрана при повороте устройства и пересоздании фрагмента.

3) Экран настроек и управление темой.

В разделе настроек реализован переключатель светлой и тёмной темы. Выбранное значение сохраняется во ViewModel и применяется ко всему приложению через AppCompatActivity, что демонстрирует работу MVVM для хранения и применения глобальных настроек.

4) Сохранение состояния при изменении конфигурации.

Так как все данные хранятся во ViewModel, при повороте экрана состояние приложения сохраняется. Пользовательские данные и выбранные настройки не теряются, что положительно влияет на удобство использования приложения.

5) Логирование жизненного цикла ViewModel.

В ViewModel добавлено логирование основных этапов жизненного цикла - создания и уничтожения объектов. Это позволяет контролировать корректность работы архитектуры MVVM и отслеживать момент освобождения ресурсов.

Вывод

В результате была реализована архитектура MVVM, обеспечивающая разделение логики и UI, корректное сохранение состояния при повороте экрана и реактивное обновление интерфейса через LiveData. Использование ViewModel позволило надежно хранить данные профиля и настройки темы, а логирование жизненного цикла упростило контроль и отладку работы приложения.

Задача 3 - Загрузка сообщений из API и Room для офлайн доступа

ТЗ

Необходимо реализовать загрузку списка сообщений из внешнего API с последующим сохранением данных в локальную базу Room для обеспечения офлайн-доступа. Сетевое взаимодействие должно выполняться с использованием Retrofit и корутин. Отображение сообщений требуется реализовать на экране «Лента» с помощью RecyclerView. Репозиторий должен выступать единой точкой доступа ViewModel к данным из сети и базы

Решение

1) Репозиторий сообщений.

В проекте реализован класс репозитория, отвечающий за получение и хранение сообщений. Он объединяет работу с удалённым API и локальной базой данных Room, предоставляя ViewModel единый механизм доступа к данным. Такой подход позволяет централизовать логику работы с источниками данных и реализовать принцип единого источника истины.

2) Сетевое взаимодействие через Retrofit и корутины.

Для обращения к внешнему API используется Retrofit, настроенный на работу с корутинами. Это обеспечивает асинхронную загрузку данных без блокировки главного потока и упрощает обработку сетевых ответов и возможных ошибок

3) Отображение ленты сообщений.

Список сообщений выводится на экране «Лента» с использованием RecyclerView. Каждый элемент списка содержит основную информацию о сообщении, что позволяет пользователю удобно просматривать загруженный контент.

4) Локальное хранение данных.

После успешного получения сообщений из сети данные сохраняются в локальную базу данных Room. Это позволяет повторно использовать уже загруженную информацию и снижает количество сетевых запросов.

5) Поддержка офлайн-режима.

При отсутствии подключения к интернету репозиторий автоматически загружает

данные из локальной базы. Такой механизм обеспечивает стабильную работу приложения и доступность информации в офлайн-режиме

6) Ручное обновление данных.

На экране ленты реализована кнопка обновления, которая инициирует повторную загрузку данных через ViewModel. При этом данные обновляются как в интерфейсе, так и в локальной базе, предоставляя пользователю контроль над актуальностью информации

Вывод

Реализация репозитория в связке с Retrofit, Room и корутинами обеспечила единый и надёжный источник данных для приложения. Лента сообщений корректно работает как онлайн, так и офлайн, поддерживает ручное обновление и эффективно использует локальное хранилище, что повышает стабильность, производительность и удобство работы для пользователя.

Задача 4 - Улучшение интерфейса и фоновой синхронизации

ТЗ

Требуется доработать пользовательский интерфейс приложения и расширить его функциональность за счёт внедрения элементов Material Design. Необходимо реализовать интерактивные элементы списка сообщений, включая отображение аватара, имени автора и возможность ставить отметку «лайк». Также требуется настроить фоновую синхронизацию сообщений с использованием WorkManager и отправку уведомлений при успешном обновлении данных. Приложение должно корректно работать в фоновом режиме, запрашивать необходимые разрешения при запуске и корректно реагировать на изменение состояния сети, сохраняя архитектуру MVVM.

Решение

Кастомный список сообщений:

Цветные аватары с первой буквой имени

Имя пользователя, текст сообщения

Лайки/дизлайки с изменяемыми счетчиками

Счетчик просмотров Material Design компоненты:

MaterialCardView для каждого сообщения

FloatingActionButton для обновления данных

AppBarLayout с Toolbar

Лайки/дизлайки:

Клик по иконке меняет состояние (filled/border)

Счетчики увеличиваются/уменьшаются

Взаимоисключающие (лайк убирает дизлайк и наоборот)

WorkManager:

Фоновая синхронизация каждые 15 минут

Работает только при наличии интернета

Сохраняет пользовательские лайки/дизлайки при обновлении

Уведомления:

NotificationChannel для Android 8+

Уведомление "Новые данные получены" после синхронизации

Работает в фоновом режиме

Запросы

При входе в приложение запрашивает доступ на получения списка контактов и отправка уведомлений

Вывод

Внедрение компонентов Material Design и фоновой синхронизации через WorkManager значительно улучшило пользовательский опыт. Приложение теперь обладает современным адаптивным интерфейсом с интерактивными элементами, корректно работает в фоновом режиме и своевременно уведомляет пользователя об обновлениях данных, сохраняя при этом целостность архитектуры MVVM и реакцию на изменения состояния сети.

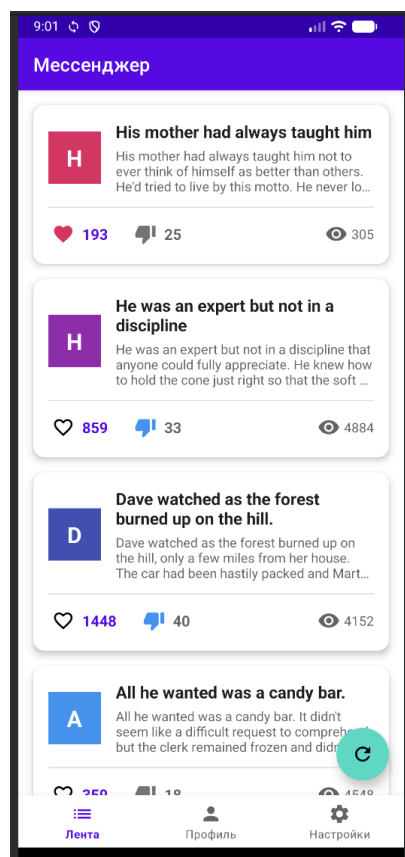


Рисунок 3 - Экран Ленты

XML архитектура проекта

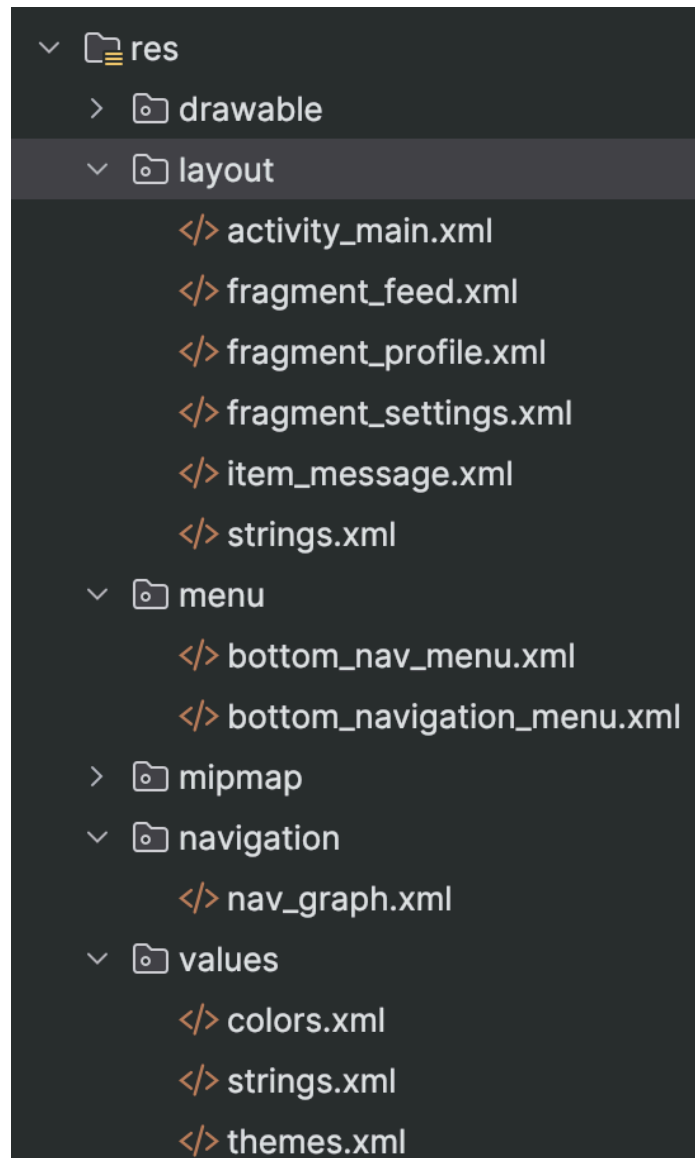


Рисунок 4 - XML файлы

Вывод:

XML-файлы структурируют интерфейс и навигацию приложения: layout-файлы отвечают за экраны и элементы списка, menu - за нижнюю навигацию, navigation - за логику переходов, values - за ресурсы оформления (цвета, строки, темы)

Листинг кода

[AppDatabase.kt](#)

```
package com.example.messenger.data

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase
import androidx.room.TypeConverters

@Database(entities = [Message::class], version = 4, exportSchema =
false)
@TypeConverters(Converters::class)
abstract class AppDatabase : RoomDatabase() {
    abstract fun messageDao(): MessageDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "main_db3"
                )
                    .fallbackToDestructiveMigration()
                    .build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

Этот класс отвечает за создание и управление локальной базой данных приложения с использованием Room. Он предоставляет единый экземпляр базы данных, содержит DAO для работы с сообщениями и обеспечивает безопасный доступ к данным

[Message.kt](#)

```
package com.example.messenger.data

import androidx.room.Entity
import androidx.room.PrimaryKey
import androidx.room.TypeConverter
import androidx.room.TypeConverters
import com.google.gson.Gson
import com.google.gson.annotations.SerializedName

data class Reactions(
    val likes: Int = 0,
    val dislikes: Int = 0
)

class Converters {
    @TypeConverter
    fun fromReactions(reactions: Reactions?): String? {
        return reactions?.let { Gson().toJson(it) }
    }

    @TypeConverter
    fun toReactions(reactionsString: String?): Reactions? {
        return reactionsString?.let { Gson().fromJson(it,
Reactions::class.java) }
    }
}

@Entity(tableName = "messages")
@TypeConverters(Converters::class)
data class Message(
    @PrimaryKey
    val id: Int,

    @SerializedName("title")
    val name: String,

    @SerializedName("body")
    val email: String,

    @SerializedName("userId")
    val body: String,

    @SerializedName("reactions")
    val reactions: Reactions = Reactions(),

    @SerializedName("views")
    val views: Int = 0,
```

```

    val isLiked: Boolean = false,
    val isDisliked: Boolean = false,

    val userLikes: Int = 0,
    val userDislikes: Int = 0
)

```

Код описывает сущность Message для Room с полем reactions, которое сериализуется в JSON через конвертер. Reactions хранит лайки и дизлайки, а Message включает данные сообщения, просмотры и пользовательские реакции

[MessageApiService.kt](#)

```

package com.example.messenger.data

import retrofit2.http.GET
import retrofit2.http.Query

interface MessageApiService {
    @GET("posts")
    suspend fun getMessages(
        @Query("limit") limit: Int,
        @Query("skip") skip: Int
    ): PostsResponse
}

```

Интерфейс Retrofit MessageApiService с методом getMessages для получения списка постов с пагинацией через limit и skip

[MessageDao.kt](#)

```

package com.example.messenger.data

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
import kotlinx.coroutines.flow.Flow

@Dao
interface MessageDao {

```

```

@Query("SELECT * FROM messages")
suspend fun getAllMessages(): List<Message>

@Query("SELECT * FROM messages")
fun getAllMessagesFlow(): Flow<List<Message>>

@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insertMessages(messages: List<Message>)

@update
suspend fun updateMessage(message: Message)

@Query("DELETE FROM messages")
suspend fun deleteAll()
}

```

Интерфейс для работы с таблицей messages в Room, позволяющий читать, обновлять, вставлять и удалять сообщения, включая поддержку потоков данных через Flow

[MessageRepository.kt](#)

```

package com.example.messenger.data

import android.util.Log
import kotlinx.coroutines.flow.Flow

class MessageRepository(private val api: MessageApiService,
private val db: AppDatabase) {
    private val dao = db.messageDao()

    fun getMessagesFlow(): Flow<List<Message>> {
        return dao.getAllMessagesFlow()
    }

    suspend fun fetchMessages(limit: Int): List<Message> {
        return try {
            Log.d("MessageRepository", "Загрузка из API:
limit=$limit")

            val response = api.getMessages(limit, 0)
            Log.d("MessageRepository", "Получено
${response.posts.size} сообщений из API")

            val existingMessages = dao.getAllMessages().associateBy
{ it.id }

```

```

        val updatedMessages = response.posts.map { newMsg ->
            val existing = existingMessages[newMsg.id]
            if (existing != null) {
                newMsg.copy(
                    isLiked = existing.isLiked,
                    isDisliked = existing.isDisliked,
                    userLikes = existing.userLikes,
                    userDislikes = existing.userDislikes
                )
            } else {
                newMsg
            }
        }

        dao.deleteAll()
        dao.insertMessages(updatedMessages)

        updatedMessages
    } catch (e: Exception) {
        Log.e("MessageRepository", "Ошибка загрузки из API,
используем кэш", e)

        val cachedMessages = dao.getAllMessages()
        Log.d("MessageRepository", "Загружено
${cachedMessages.size} сообщений из БД")

        cachedMessages
    }
}

suspend fun toggleLike(message: Message) {
    val newIsLiked = !message.isLiked
    val newUserLikes = if (newIsLiked) message.userLikes + 1
else message.userLikes - 1

    val updatedMessage = message.copy(
        isLiked = newIsLiked,
        isDisliked = false,
        userLikes = newUserLikes,
        userDislikes = if (message.isDisliked)
message.userDislikes - 1 else message.userDislikes
    )
    dao.updateMessage(updatedMessage)
}

suspend fun toggleDislike(message: Message) {
    val newIsDisliked = !message.isDisliked

```

```

        val newUserDislikes = if (newIsDisliked)
message.userDislikes + 1 else message.userDislikes - 1

        val updatedMessage = message.copy(
            isDisliked = newIsDisliked,
            isLiked = false,
            userDislikes = newUserDislikes,
            userLikes = if (message.isLiked) message.userLikes - 1
else message.userLikes
        )
        dao.updateMessage(updatedMessage)
    }
}

```

управляет данными сообщений, комбинируя API и локальную базу. Он загружает сообщения с сервера, синхронизирует их с локальными реакциями, кэширует в Room и предоставляет поток (Flow) для наблюдения. Также реализованы методы для переключения лайка и дизлайка с корректным обновлением счётчиков и состояния пользователя

[RetrofitInstance.kt](#)

```

package com.example.messenger.data

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitInstance {
    private const val BASE_URL = "https://dummyjson.com/"

    val api: MessageApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(MessageApiService::class.java)
    }
}

```

RetrofitInstance — это объект-одиночка, который настраивает и предоставляет готовый Retrofit для работы с API. Он задаёт базовый URL. Подключает конвертер Gson для автоматической сериализации и десериализации JSON, и создаёт экземпляр MessageApiService, через который можно вызывать сетевые запросы к серверу.

Вывод по работе

В процессе выполнения курсовой работы было разработано Android-приложение, основанное на архитектуре MVVM с применением компонентов Material Design и актуальных инструментов платформы Android, таких как Room, Retrofit, Navigation Component, LiveData и ViewModel. Реализована полноценная работа с данными, включающая загрузку информации из сети, её локальное хранение и последующее отображение в пользовательском интерфейсе с поддержкой интерактивных функций. Разработанное приложение может быть использовано как базовая платформа для дальнейшего расширения функционала, включая реализацию фоновой синхронизации, системы уведомлений и дополнительных возможностей работы с профилем. Результаты работы подтверждают освоение основных принципов и практик разработки современных Android-приложений